

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Penganter Mikrokontroler

Christopher Joenathan Hindarto - 5024241028

07 November 2026

1 Pendahuluan

1.1 Latar Belakang

Pengembangan sistem tertanam (*embedded system*) modern menuntut pemahaman mendalam mengenai arsitektur perangkat keras serta pemilihan *software stack* dan lingkungan pengembangannya. Perkembangan platform mikrokontroler seperti STM32 Blackpill berbasis ARM Cortex-M4 dan ESP32 berbasis Xtensa LX6 dual-core menawarkan fleksibilitas eksekusi program yang bervariasi. Penggunaan IDE terintegrasi seperti Visual Studio Code dengan ekstensi PlatformIO mempermudah pengelolaan berbagai *framework* (seperti Arduino dan ESP-IDF) dalam satu ekosistem terpadu. Penggunaan *framework* Arduino menyajikan abstraksi tingkat tinggi untuk kemudahan prototipe, sedangkan *framework* native seperti ESP-IDF menyajikan kontrol berdaya tinggi hingga ke tingkat *task scheduling* FreeRTOS dan registri sistem.

Dalam perancangan antarmuka perangkat keras, pengelolaan sinyal Masukan/Keluaran Serbaguna atau *General Purpose Input/Output* (GPIO) memainkan peranan utama. Namun, transisi sinyal digital pada input mekanis seperti tombol tekan (*pushbutton*) tidak terlepas dari fenomena getaran fisik kontak mekanik (*bouncing*) yang memicu deret pulsa transisi cepat HIGH-LOW sebelum sinyal benar-benar stabil. Jika dibiarkan tanpa penanganan, hal ini terbaca sebagai penekanan ganda oleh mikroprosesor. Selain itu, kebutuhan pembacaan sinyal kontinu menuntut pemanfaatan periferal *Analog-to-Digital Converter* (ADC). Oleh karena itu, praktikum ini dilaksanakan untuk memberikan pemahaman fundamental mengenai integrasi arsitektur board, manajemen GPIO digital-analog, serta penanganan *debouncing* sinyal sebelum membangun sistem kontrol dan IoT yang lebih kompleks.

1.2 Dasar Teori

GPIO dapat dikonfigurasikan ke dalam berbagai mode operasi dasar, yaitu *digital output*, *digital input*, serta *analog input*. Pada mode *digital input*, keterikatan sinyal terhadap noise eksternal atau status menggantung (*floating*) dapat dicegah dengan mengimplementasikan mekanisme resistor *pull-up* atau *pull-down*, baik secara eksternal pada papan reka (*breadboard*) maupun memanfaatkan fungsi *pull internal* pada mikrokontroler. Penerapan konfigurasi ini menjamin kepastian logika tegangan (HIGH atau LOW) saat saklar mekanis tidak diaktifkan. Untuk mengatasi masalah *bouncing* pada input mekanis, teknik penyaringan sinyal secara komputasional (*debouncing*) wajib diimplementasikan dengan memvalidasi durasi kestabilan sinyal menggunakan pewaktu sistem (seperti fungsi `millis()` atau pewaktu *tick* FreeRTOS) sebelum mengonfirmasi status logis tombol.

Berbeda dengan mode digital yang hanya membedakan dua kondisi status tegangan diskrit (HIGH/LOW), mode *analog input* menggunakan *Analog-to-Digital Converter* (ADC) untuk membaca besaran tegangan analog yang berubah secara kontinu. ADC mengubah tegangan analog (seperti keluaran sensor resistif pembagi tegangan LDR pada rentang 0–3.3V) menjadi representasi nilai kuantisasi digital. Sebagai contoh, resolusi ADC 12-bit pada ESP32 membagi rentang tegangan referensi menjadi $2^{12} = 4096$ tingkatan langkah (*step/levels*), sehingga menghasilkan nilai mentah digital dari 0 hingga 4095. Hal ini memungkinkan mikrokontroler untuk mengukur dan memproses besaran fisis secara lebih presisi.

2 Tugas Pendahuluan

2.1 Implementasi Program Matrix Pushbutton 3x3 ke LED Matrix 3x3 dengan Debouncing

Berikut adalah kode program C++ berbasis Framework Arduino pada ESP32 yang mengimplementasikan pemindaian matriks tombol 3x3 untuk mengendalikan LED matriks 3x3 dengan fitur *software debouncing* terintegrasi pada setiap tombol:

Listing 1: Kode Program Matrix 3x3 Button to LED with Debounce pada ESP32

```
#include <Arduino.h>

const int NUM_ROWS = 3;
const int NUM_COLS = 3;

// Pin Assignment Matrix Tombol
const int btnRows[NUM_ROWS] = {12, 14, 27};
const int btnCols[NUM_COLS] = {26, 25, 33};

// Pin Assignment Matrix LED
const int ledRows[NUM_ROWS] = {19, 18, 5};
const int ledCols[NUM_COLS] = {17, 16, 4};

// State Matrix untuk LED (true = ON, false = OFF)
bool ledState[NUM_ROWS][NUM_COLS] = {
  {false, false, false},
  {false, false, false},
  {false, false, false}
};

// Variabel Debouncing per Tombol
unsigned long lastDebounceTime[NUM_ROWS][NUM_COLS];
int buttonStableState[NUM_ROWS][NUM_COLS];
int buttonFlickerState[NUM_ROWS][NUM_COLS];
const unsigned long DEBOUNCE_DELAY = 50; // ms

void setup() {
  Serial.begin(115200);

  // Inisialisasi Pin LED Matrix
  for (int r = 0; r < NUM_ROWS; r++) {
    pinMode(ledRows[r], OUTPUT);
    digitalWrite(ledRows[r], LOW);
  }
  for (int c = 0; c < NUM_COLS; c++) {
    pinMode(ledCols[c], OUTPUT);
    digitalWrite(ledCols[c], HIGH); // Katoda HIGH -> LED Mati
  }

  // Inisialisasi Pin Button Matrix
  for (int r = 0; r < NUM_ROWS; r++) {
    pinMode(btnRows[r], OUTPUT);
    digitalWrite(btnRows[r], HIGH); // Idle HIGH
  }
```

```
  for (int c = 0; c < NUM_COLS; c++) {
    pinMode(btnCols[c], INPUT_PULLUP); // Pull-up Internal Kolom
  }

  // Inisialisasi Array Debounce State
  for (int r = 0; r < NUM_ROWS; r++) {
    for (int c = 0; c < NUM_COLS; c++) {
      lastDebounceTime[r][c] = 0;
      buttonStableState[r][c] = HIGH;
      buttonFlickerState[r][c] = HIGH;
    }
  }
}

void scanButtons() {
  for (int r = 0; r < NUM_ROWS; r++) {
    // Aktifkan baris aktif (LOW)
    digitalWrite(btnRows[r], LOW);

    for (int c = 0; c < NUM_COLS; c++) {
      int reading = digitalRead(btnCols[c]);

      // Deteksi perubahan sinyal sementara (flicker)
      if (reading != buttonFlickerState[r][c]) {
        lastDebounceTime[r][c] = millis();
        buttonFlickerState[r][c] = reading;
      }

      // Validasi kestabilan sinyal setelah DEBOUNCE_DELAY
      if ((millis() - lastDebounceTime[r][c]) > DEBOUNCE_DELAY) {
        if (reading != buttonStableState[r][c]) {
          buttonStableState[r][c] = reading;

          // Toggle status LED saat tombol ditekan (transisi HIGH -> LOW)
          if (buttonStableState[r][c] == LOW) {
            ledState[r][c] = !ledState[r][c];
            Serial.printf("Tombol [%d][%d] Ditekan! State LED [%d][%d]: %s\n",
                          r, c, r, c, ledState[r][c] ? "ON" : "OFF");
          }
        }
      }
    }

    // Kembalikan baris ke kondisi Idle (HIGH)
    digitalWrite(btnRows[r], HIGH);
  }
}

void refreshLEDs() {
  // Multiplexing Display LED Matrix
  for (int r = 0; r < NUM_ROWS; r++) {
    digitalWrite(ledRows[r], HIGH); // Nyalakan baris r
    for (int c = 0; c < NUM_COLS; c++) {
      if (ledState[r][c]) {
```

```
99          digitalWrite(ledCols[c], LOW);  // Ground-kan kolom c jika ON
100       } else {
101         digitalWrite(ledCols[c], HIGH); // Matikan kolom c
102       }
103     }
104     delayMicroseconds(500); // Tampilan singkat
105     digitalWrite(ledRows[r], LOW);  // Matikan baris r
106
107     // Reset kolom
108     for (int c = 0; c < NUM_COLS; c++) {
109       digitalWrite(ledCols[c], HIGH);
110     }
111   }
112 }
113
114 void loop() {
115   scanButtons();
116   refreshLEDs();
117 }
```

2.2 Jawaban Pertanyaan Analisis

1. Urgensi Debouncing pada Rangkaian Matrix Pushbutton

Pertanyaan: Analisis mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton ini, walaupun pada satu waktu hanya satu tombol yang biasanya ditekan, kaitkan dengan penyebab bouncing pada C.5 Debouncing!

Jawaban:

Debouncing tetap mutlak diperlukan karena fenomena *bouncing* berasal dari **karakteristik mekanis fisik internal tombol itu sendiri** (yaitu getaran elastis plat kontak platina saat bersentuhan atau terlepas), bukan akibat penekanan beberapa tombol sekaligus. Dalam skema pemindaian matriks (*matrix scanning*), mikrokontroler melakukan pembacaan logika pin ribuan kali dalam satu detik. Saat satu tombol ditekan, kontak mekanis bergetar dan menghasilkan pulsa transisi sinyal HIGH-LOW berulang secara acak selama beberapa milidetik. Tanpa *debouncing*, pemindaian cepat mikrokontroler akan mengidentifikasi getaran tersebut sebagai **banyak peristiwa penekanan tombol terpisah (*multiple triggers*)**. Akibatnya, status logika LED yang ingin dikontrol secara sakelar ganda (*toggle*) akan berganti secara acak dan tidak stabil.

2. Analisis Konfigurasi Pull-up/Pull-down pada Kolom Tombol

Pertanyaan: Analisis konfigurasi pull-up atau pull-down (sesuai rancangan Anda) yang digunakan pada pembacaan kolom tombol, apa kondisi (HIGH/LOW) pin tersebut saat tombol **tidak** ditekan, dan mengapa kondisi tersebut tidak boleh dibiarkan *floating*?

Jawaban:

- **Konfigurasi yang Digunakan:** Pull-up internal (`INPUT_PULLUP`) pada pin kolom ESP32.
- **Kondisi Pin Saat TIDAK Ditekan:** Berada pada logika **HIGH** (3.3V).

- **Alasan Tidak Boleh *Floating*:** Jika pin kolom dibiarkan menggantung (*floating*) tanpa resistor *pull*, pin tersebut memiliki impedansi sangat tinggi dan berfungsi seperti antena yang rentan menangkap *electromagnetic noise* dari lingkungan sekitar. Sinyal tegangan pada GPIO akan meloncat-loncat secara acak antara HIGH dan LOW. Hal ini menyebabkan mikrokontroler memproses penekanan tombol palsu (*ghosting / false triggering*) meskipun tombol tidak ditekan secara fisik.

3. Perbedaan Pembacaan GPIO Digital Input vs Analog Input (ADC)

Pertanyaan: Andaikan salah satu LED pada matrix ini digantikan dengan LDR yang dibaca melalui ADC (seperti Percobaan 5), analisis perbedaan mendasar antara membaca GPIO sebagai **digital input** (tombol, hanya HIGH/LOW) dan sebagai **analog input** (ADC), mengapa ADC dapat merepresentasikan lebih dari dua kondisi?
Jawaban:

1. **Digital Input:** Hanya mengenali **dua kondisi diskrit** (logika biner: LOW/0 atau HIGH/1). Komparator internal GPIO membandingkan tegangan input dengan nilai batas (*threshold*); tegangan di bawah ambang terbaca sebagai LOW, dan di atas ambang terbaca sebagai HIGH.

2. **Analog Input (ADC):** Mampu membaca dan memetakan **banyak kondisi nilai kontinu** pada seluruh rentang tegangan input analog.

3. **Alasan ADC Dapat Membaca Lebih dari 2 Kondisi:** Hal ini dimungkinkan oleh adanya periferal **kuantisasi resolusi** pada ADC (misalnya resolusi 12-bit pada ESP32). Resolusi 12-bit membagi rentang tegangan acuan (0V hingga 3.3V) menjadi $2^{12} = 4096$ tingkatan langkah (*step*). Perubahan resistansi pada sensor analog (seperti LDR akibat tingkat pencahayaan) merubah tegangan masukan secara linier, dan ADC mengonversi nilai tersebut menjadi angka digital mentah berentang dari **0 hingga 4095**.